

CSAN 2026 · CURSO PRE-CONGRESO · DÍA 2

# Diseño y ejecución de experimentos conductuales

**Juan Kamienkowski · Octavio Castro · Ramón Igarreta · Gustavo Verón**

Laboratorio de Inteligencia Artificial Aplicada · ICC, UBA–CONICET



HOY · DE 9:30 A 17:00

# Ejecutar, sincronizar y analizar

| Hora  |                                             |
|-------|---------------------------------------------|
| 9:30  | Ejecución cruzada                           |
| 10:05 | Buenas prácticas y validación: timing       |
| 10:50 | Café                                        |
| 11:05 | Entornos de ejecución                       |
| 11:25 | Sincronización y hardware, con osciloscopio |
| 12:10 | Extender PsychoPy con código                |
| 12:30 | Almuerzo                                    |

| Hora  |                                            |
|-------|--------------------------------------------|
| 13:30 | Código: una marca en el momento justo      |
| 14:00 | Revisión de resultados y archivos de datos |
| 14:30 | Leer y analizar los datos de la clase      |
| 15:40 | Café                                       |
| 15:55 | El efecto Stroop de la clase               |
| 16:20 | Charla sobre sus proyectos y cierre        |
| 17:00 | Fin del curso                              |

# Ejecución cruzada

1. Cada pareja deja su experimento abierto en el Builder, en su computadora
2. En ronda: cada pareja corre los de la anterior y la siguiente (B corre A y C; A corre la última y B)
3. Vayan a esas computadoras y córranlo con **su** número, en serio
4. No hay que subir nada: el CSV queda en la [data/](#) de lxs autorxs

Después del almuerzo, cada pareja abre los datos que le generaron.

JUAN

# Buenas prácticas y validación

Timing: lo que se pide, lo que la computadora hace y lo que se ve

## TIMING

**Si piden que la palabra dure 200 ms,  
¿dura 200 ms?**

El archivo registra 0.2 s: es la duración pedida, no la medida.

# Exactitud y precisión

## Exactitud

¿Está centrado en el valor pedido?

$$\text{sesgo} = \bar{d} - d^*$$

Si el retraso es **constante**, puede restarse y las medidas quedan corregidas

Retraso constante  $c$ :  $\text{Var}(A + c) = \text{Var}(A)$

$$(A + c) - (B + c) = A - B \rightarrow \text{el efecto no cambia}$$

$d$ : duración medida en cada trial ·  $d^*$ : la pedida ·  $A$  y  $B$ : los tiempos de reacción de dos condiciones

# Exactitud y precisión

## Exactitud

¿Está centrado en el valor pedido?

$$\text{sesgo} = \bar{d} - d^*$$

Si el retraso es **constante**, puede restarse y las medidas quedan corregidas

## Precisión

¿Cuánto varía de un trial a otro?

$$s = \sqrt{[ \Sigma (d_i - \bar{d})^2 / (n - 1) ]}$$

Un retraso con **dispersión grande** es un problema porque no se puede corregir

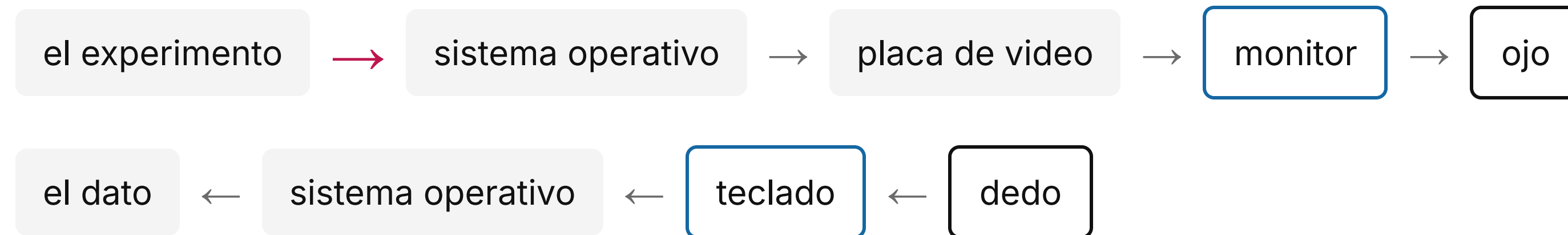
Retraso variable x:  $\text{Var}(A + x) = \text{Var}(A) + \text{Var}(x) + 2 \text{Cov}(A, x)$

Aunque x no dependa de A ( $\text{Cov} = 0$ ), la varianza crece en  $\text{Var}(x) \rightarrow$  el ruido crece

x: el retraso de cada trial, distinto cada vez

## TIMING

# La cadena de retrasos



Cada flecha suma retraso. **PsychoPy controla la primera; el eslabón débil casi nunca es el software.**

# El monitor pone el límite

## 16.7 ms

= 1 / 60 Hz, lo que dura un frame

- Solo existen múltiplos de 1 / f: 16.7, 33.3, 50 ms...; **20 ms no existe**
- Se dibuja de arriba hacia abajo, también en LCD: abajo aparece casi un frame después
- El procesamiento de imagen de monitores y TVs suma 20–30 ms variables: el modo *Game* lo apaga

# Pedir en frames

## En el Builder

Stop → **duration (frames)** → 12

$$n = \text{duración} \times f = 0.2 \text{ s} \times 60 \text{ Hz} = 12$$

## Y al empezar

Medir la frecuencia real. Nunca asumir 60 Hz:  
hay pantallas a 59.94, 120 o 144

```
# Lo que hace por dentro
for frame in range(12):
    palabra.draw()
    win.flip()
```

`win.flip()` espera al próximo refresco del monitor: por eso contar frames es exacto.

# Teclado y audio

| Dispositivo                | Retraso                                        |
|----------------------------|------------------------------------------------|
| Teclado USB común          | 4 a 25 ms, según el modelo, y no siempre igual |
| Caja de botones            | Menos de 1 ms, con reloj propio                |
| Audio con PsychoPy (PTB)   | Del orden del ms, si se agenda                 |
| Audio con librerías viejas | Hasta 100 ms                                   |

Para tiempos de reacción críticos, **caja de botones**; para un sonido crítico, **medirlo**.

# Teclas apretadas antes de tiempo

```
respuesta.clearEvents() # tirar lo que ya se apretó  
win.flip() # aparece el estímulo  
respuesta.clock.reset() # el reloj arranca acá
```

- El Builder lo hace solo cuando el teclado arranca con el estímulo
- Un RT de 3 ms en el archivo es alguien que se adelantó
- En el análisis, el piso de 200 ms atrapa lo que se escape

# Qué midió el *timing mega-study*

En laboratorio

**< 1 ms**

de precisión media en estímulo visual, audio y respuesta, para PsychoPy, Psychtoolbox, Presentation y E-Prime

Online, tiempos de respuesta

**< 3.5 ms**

de precisión con PsychoPy en todos los navegadores probados; más variable en todo lo demás

Bridges, Pitiot, MacAskill y Peirce (2020). The timing mega-study. *PeerJ*, 8, e9414. Medido con hardware externo.

## TIMING

# Frames perdidos

- Frame perdido: un intervalo entre flips mayor a  $1.5 / f$  (a 60 Hz, más de 25 ms); el estímulo dura **16.7 ms más**
- Culpables: otros programas, el antivirus, la ventana chica, imágenes que se cargan justo cuando hay que mostrarlas
- Remedio barato: pantalla completa, nada más abierto, estímulos precargados

El problema serio: que se pierdan más en una condición que en otra, **porque la diferencia de duración queda confundida con la condición.**

# Midan su máquina

1. Coder →

[dia2/medir\\_maquina/medir\\_maquina.py](#) →

Run

2. Diez segundos gira una barra: no toquen nada

3. Anoten los números

4. `FULLSCREEN = False` y otra vez

¿A cuántos Hz está la pantalla?

¿Qué cambió en ventana?

¿Qué duraciones se pueden pedir y cuáles no existen?

¿Se perdieron frames? ¿Qué había abierto?

# Buenas prácticas

- Pantalla completa, siempre
- Nada más abierto; wifi y avisos apagados
- Medir la frecuencia real al empezar
- Duraciones críticas en frames
- Precargar imágenes y sonidos
- Guardar más de lo que parece necesario
- Correrse a uno mismo, y mirar los datos
- Reportar versión, sistema, monitor y Hz
- Si el efecto es de pocos ms: medir con hardware

JUAN

# Entornos de ejecución

El lugar donde corre el experimento limita qué se puede medir

# Laboratorio, online y con otro equipo

| Dónde           | Qué se controla                 | Precisión                                 | Sirve para                   |
|-----------------|---------------------------------|-------------------------------------------|------------------------------|
| Laboratorio     | Monitor, teclado, luz, silencio | Menos de 1 ms en software                 | Efectos chicos               |
| Online          | Casi nada                       | De ms a decenas de ms                     | Efectos grandes, mucha gente |
| Con otro equipo | Lo que manda el equipo          | La de la sincronización entre los relojes | EEG, eyetracking, fMRI       |

El criterio: **el tamaño del efecto buscado frente a la variabilidad temporal del entorno.**

# Sincronizar con otro aparato

## EEG

El experimento manda una marca en cada evento

Lo vemos en el bloque que sigue, con el osciloscopio

---

## No derivar

Si cada trial se pasa 3 ms, en 300 trials: 0.9 s

*Non-slip timing*: rutinas de duración fija, y PsychoPy corrige sobre la marcha

---

## fMRI

Al revés: el pulso del escáner llega como una tecla

El experimento la espera: un Keyboard que acepta, por ejemplo, '5'

---

OCTAVIO

# Sincronización y hardware

Marcas TTL, medidas con un osciloscopio

# El problema de los dos relojes

PsychoPy

muestra el estímulo a los 12.480 s de su reloj

EEG

viene grabando desde hace 8241.32 s en el suyo

¿En qué punto del EEG apareció el estímulo? **Hace falta una marca en los dos a la vez.**

# Qué es una marca TTL

TTL: *Transistor-Transistor Logic*

1

cable por bit

0 o 5 V

apagado o prendido

$2^8 = 256$

códigos con 8 cables, de 0 a 255

La ventaja: entre el código y el cable **no** hay sistema operativo, buffers ni drivers.

# Por dónde sale la marca hoy

| Vía                             | Retraso                   | Comentario                                                |
|---------------------------------|---------------------------|-----------------------------------------------------------|
| Puerto paralelo                 | Despreciable              | Ya no viene en ninguna notebook                           |
| Caja USB de triggers (USB2TTL8) | $\leq 1$ ms, especificado | Se ve como puerto serie y saca 8 líneas                   |
| Arduino                         | Pocos ms, hay que medirlo | La barata                                                 |
| Pincho USB-serie genérico       | Del orden del ms          | Una sola línea: para medir timing, no para mandar códigos |

Con USB hay un buffer y un driver en el medio: **se mide una vez** en cada montaje.

# Dónde va la marca

## Mal

```
puerto.setData(codigo)  
win.flip()
```

## Bien

```
win.flip()  
puerto.setData(codigo)
```

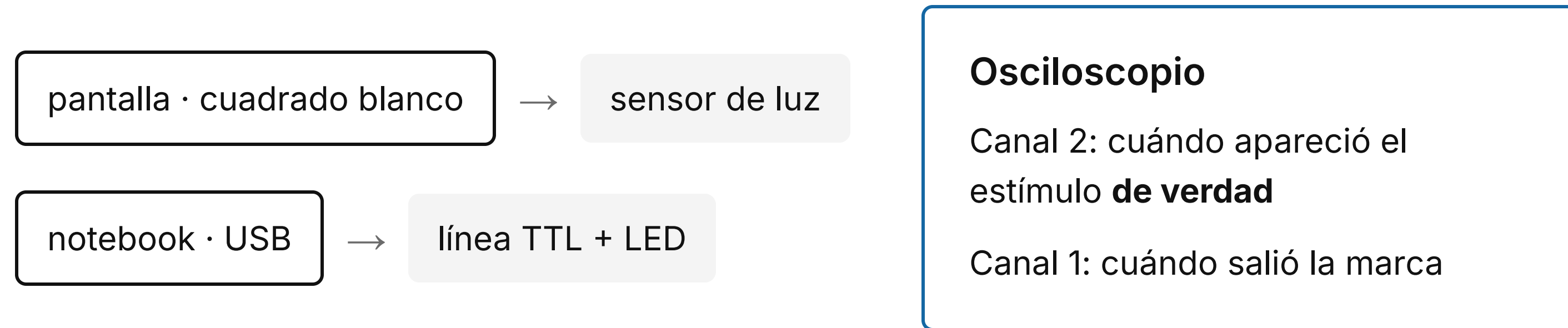
`flip()` espera al monitor, hasta 16.7 ms. Antes, la marca sale cuando el código la pide; después, **cuando el estímulo aparece**.

# Bajar la marca, y un código por condición

|                         |                                                                                         |
|-------------------------|-----------------------------------------------------------------------------------------|
| Volver a cero           | Si la marca no baja, dos marcas iguales seguidas se ven como una                        |
| Un código por condición | 11 → congruente, 21 → incongruente: el EEG se segmenta sin abrir el CSV                 |
| Guardarlo también       | Una columna <code>trigger_code</code> en el CSV: si se pierde una marca, se reconstruye |

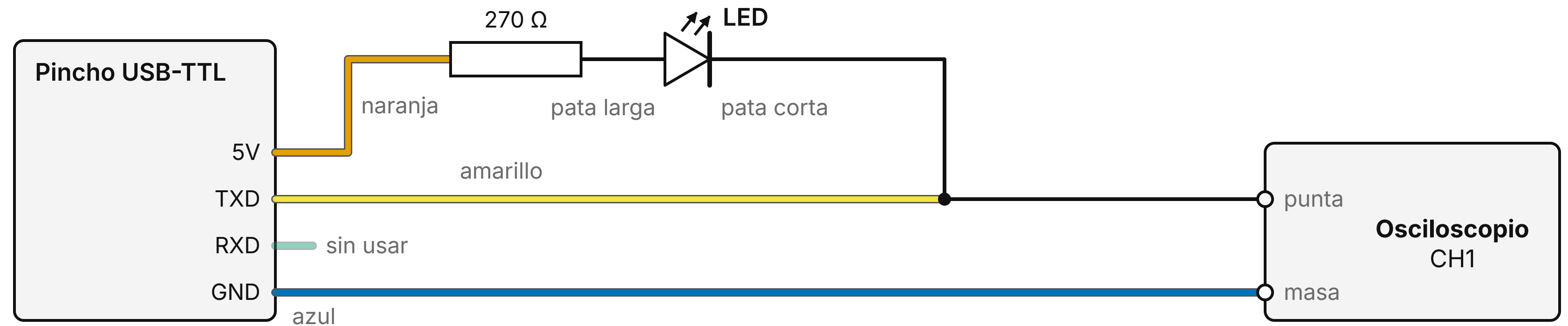
DEMO

# El montaje



La distancia entre los dos flancos **es el error**.

# La marca, en el canal 1

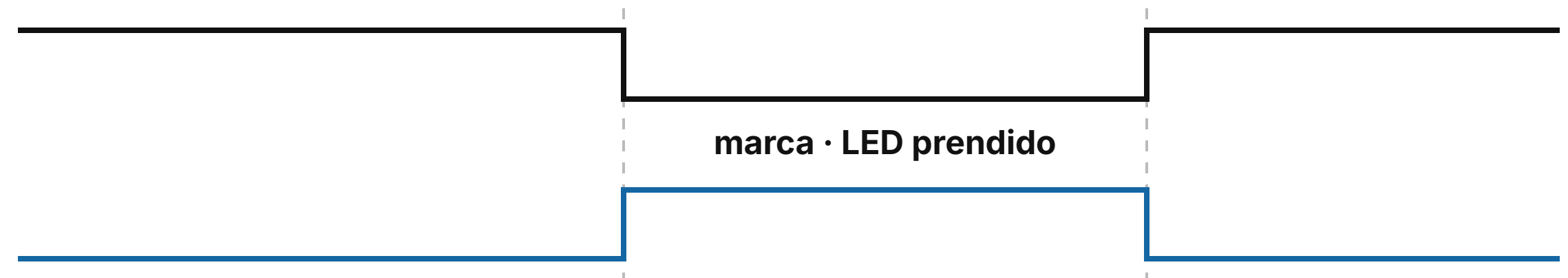


**en el cable**

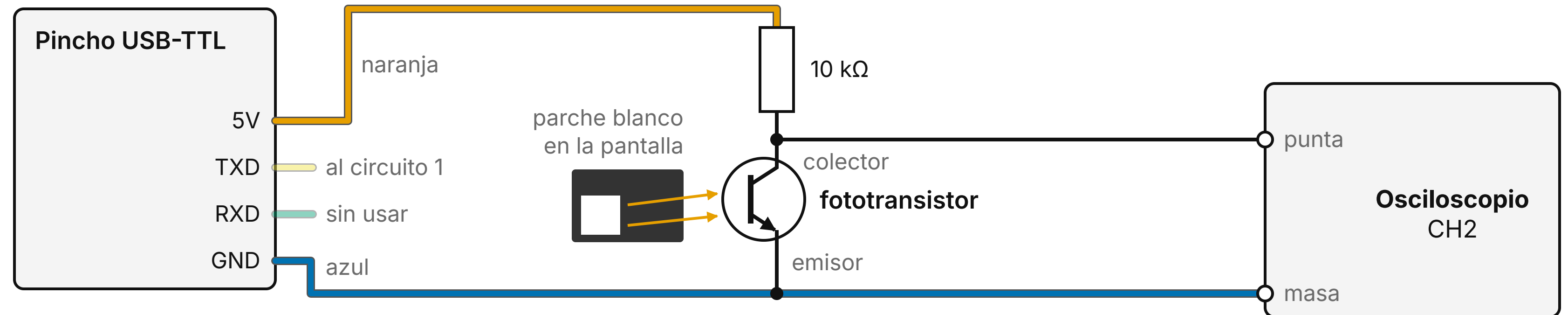
5 V en reposo, 0 V con la marca

**en el osciloscopio**

con el CH1 invertido



# El sensor de luz, en el canal 2



en el cable

5 V a oscuras, baja con luz

en el osciloscopio

con el CH2 invertido



# Marca antes o después del flip

## Corrida

## Qué se ve en el osciloscopio

Antes del flip

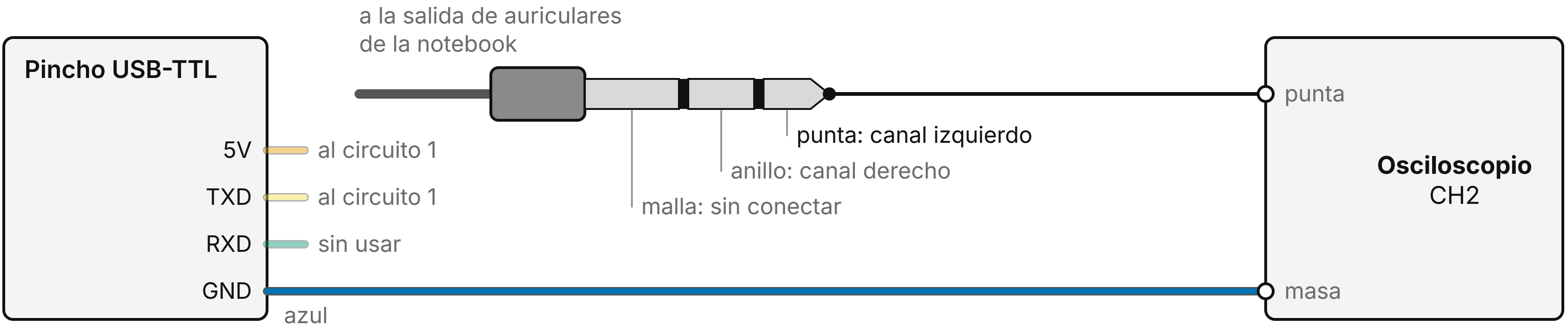
La luz llega entre 0 y 17 ms después de la marca, en un lugar distinto cada vez: una nube

Después del flip

La luz llega siempre a la misma distancia, unos pocos ms: una línea

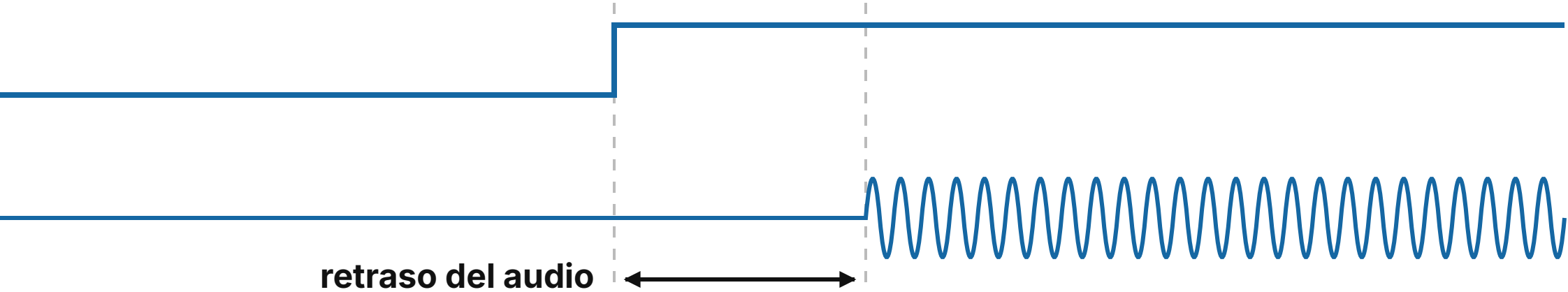
**Un retraso constante se resta; uno variable no se puede corregir después.**

# El audio, en el canal 2



**CH1: la marca**  
invertido, como en el circuito 1

**CH2: el audio**  
el bip de 1000 Hz



# ¿Cuándo suena un bip?

## Cómo se pide

## Qué se ve

play() y la marca

El sonido arranca varios ms después de la marca, y no siempre igual

Agendado

Se le pide a la placa que empiece en un instante, y la marca sale en ese instante: alineados

Con la pantalla

Como el Builder: el sonido sale con el próximo flip

El retraso del audio **no aparece en ningún archivo**: se agenda, y se mide una vez.

# Lo que medimos antes del curso

[captura: antes del flip]

[captura: después del flip]

[captura: audio]

# Otros dispositivos y protocolos

| Qué                      | Para qué                                                                                   |
|--------------------------|--------------------------------------------------------------------------------------------|
| Lab Streaming Layer      | Varios aparatos por red, cada uno con sus marcas de tiempo. Un aparato: cable; varios: LSL |
| Eyetracking (ioHub)      | Tobii, EyeLink, GazePoint, Pupil Labs; los datos van a un archivo aparte, no al CSV        |
| Cajas de botones         | Tiempos de reacción con reloj propio                                                       |
| Arduino por puerto serie | Estimulación táctil, olfativa, eléctrica; leer sensores                                    |

# Seguridad eléctrica

- Un Arduino saca **5 V**; varios amplificadores esperan **3.3 V**
- Entre la computadora y un equipo conectado a alguien: **optoacoplador**
- **Masa común**: sin las tierras unidas no se mide nada coherente
- Nada se conecta a un participante sin leer el manual y consultar a quien sepa

# Seis detalles a tener en cuenta

1. El reloj de la computadora no puede medirse a sí mismo
2. La marca va **después** del flip
3. La línea vuelve a cero
4. Un código por condición
5. El montaje se valida una vez, con un instrumento de afuera
6. Si el experimento lo decide, el experimento lo guarda

JUAN

# Extender PsychoPy con código

El Builder escribe Python: se puede leer, y se puede completar

# El Builder es un generador de scripts



- Leer el `.py` explica todo lo que el Builder hace por detrás
- Editarlo **no**: se sobrescribe la próxima vez que se corre desde el Builder

# Cada nombre del Builder es una variable

| Nombre                       | Qué es                                                             |
|------------------------------|--------------------------------------------------------------------|
| <code>palabra.status</code>  | Si el estímulo todavía no empezó, está, o ya terminó               |
| <code>respuesta.keys</code>  | La tecla que se apretó en este trial                               |
| <code>word, congruent</code> | Las columnas del archivo de condiciones, en este trial             |
| <code>trials</code>          | El bucle: <code>trials.thisN</code> , <code>trials.finished</code> |
| <code>thisExp</code>         | El archivo de datos                                                |
| <code>win</code>             | La ventana                                                         |

# Cuándo corre cada pestaña

| Pestaña           | Cuándo corre           | Para qué                            |
|-------------------|------------------------|-------------------------------------|
| Begin Experiment  | Una vez, al arrancar   | Crear contadores, definir funciones |
| Begin Routine     | Al empezar cada trial  | Preparar el trial                   |
| <b>Each Frame</b> | 60 veces por segundo   | Reaccionar mientras pasa            |
| End Routine       | Al terminar cada trial | Guardar, contar                     |
| End Experiment    | Una vez, al final      | Cerrar puertos, resumir             |

Each Frame corre **antes** de cada flip: lo que se dibuja ahí todavía no está en pantalla.

# Tres cosas que el Builder solo no hace

## Cortar una rutina

```
# Each Frame  
if respuesta.keys: continueRoutine = False
```

## Saltear una rutina

```
# Begin Routine  
if tipo_de_bloque ≠ 'practica': continueRoutine = False
```

## Contar

```
# Begin Experiment  n_correctas = 0  
# End Routine      n_correctas += respuesta.corr
```

# Si el experimento lo decide, el experimento lo guarda

Al CSV, una columna nueva

```
thisExp.addData(  
    'posicion', posicion  
)
```

Al .log, una línea con la hora

```
logging.data(  
    f'TRIGGER {codigo}'  
)
```

Todo lo que se sortea por código, se guarda. Si no, después no hay forma de saber qué vio cada quien.

# La marca, después del flip

```
# Begin Experiment
def mandar_marca(codigo):
    logging.data(f'TRIGGER {codigo}')

# Begin Routine
marca_pendiente = True

# Each Frame
if palabra.status == STARTED and marca_pendiente:
    win.callOnFlip(mandar_marca, int(trigger_code))
    marca_pendiente = False
```

`callOnFlip` no la manda ahora: la agenda para **justo después del próximo flip**.

# Una marca en el momento justo

|          |                                                                              |        |
|----------|------------------------------------------------------------------------------|--------|
| <b>A</b> | Una columna <code>trigger_code</code> en el archivo de condiciones           | 10 min |
| <b>B</b> | El componente Code, <b>debajo</b> del estímulo                               | 25 min |
| <b>C</b> | Abrir el <code>.log</code> : una línea TRIGGER por trial, pegada al estímulo | 15 min |

Para pensar: el `.log` dice que la marca y el estímulo coinciden. **¿Quién escribió las dos líneas?**

OCTAVIO

# Los archivos de datos

Los CSV que otras personas dejaron en su experimento

## DATOS

# Tres archivos por sesión

| Extensión            | Qué es                           | Cuándo se abre                       |
|----------------------|----------------------------------|--------------------------------------|
| <code>.csv</code>    | Una tabla, una fila por trial    | <b>Siempre:</b> es el que se analiza |
| <code>.log</code>    | Todo lo que pasó, con la hora    | Timing, marcas, frames perdidos      |
| <code>.psydat</code> | El experimento entero, en Python | Cuando al CSV le falta algo          |

Los crudos no se editan **nunca**. El análisis los lee y escribe copias.

# De dónde sale el nombre

**sub-07** \_ **stroop** \_ **2026-09-30\_10h15.23.456** .csv

participant      experimento      fecha y hora

- Códigos, nunca nombres y apellidos
- Ceros a la izquierda: sub-01 ... sub-27 ordena bien
- Si todxs ponen test, no se pisan, pero no se sabe quién es quién

# Cada columna es de uno de cuatro grupos

| Grupo                      | Ejemplos                                      |
|----------------------------|-----------------------------------------------|
| Del archivo de condiciones | word, congruent, correct_key                  |
| Registradas en el trial    | respuesta.keys, respuesta.rt, palabra.started |
| Estado del bucle           | trials.thisN, trials.thisIndex                |
| De la sesión               | participant, date, psychopyVersion, frameRate |

**.rt** está en **segundos**: 0.581 son 581 ms      **todo** cuenta **desde 0**

# 36 trials, 38 filas

|         |                   |
|---------|-------------------|
| fila 0  | instrucciones     |
| 1 a 36  | <b>los trials</b> |
| fila 37 | despedida         |

Cada rutina **fuera de un bucle** escribe su fila. Con bloques, cada bloque también.

```
trials = trials[trials['respuesta.rt'].notna()]
```

# Control de calidad, antes de analizar

- Cada participante tiene los trials esperados
- Un archivo por persona, y participant distinto en cada uno
- La misma psychopyVersion; frameRate cerca de 60, 120 o 144
- trials.thisIndex: cada condición la misma cantidad de veces
- Aciertos ni en el azar ni en 100 % para todxs
- Ningún RT negativo ni más largo de lo que la tarea permite

Si algo falla, el problema es del **experimento**, no del análisis.

PRÁCTICA DE A DOS

# Los datos de la clase

Consigna 6 · el Stroop mínimo que corrieron ayer a la mañana

# Cinco partes, en orden

| Parte | Qué                                                 | Tiempo |
|-------|-----------------------------------------------------|--------|
| 1     | Mirar la tabla antes de calcular nada               | 10 min |
| 2     | Describir: RT medio, mediano y aciertos por persona | 15 min |
| 3     | Graficar las dos distribuciones                     | 15 min |
| 4     | ¿Hay efecto Stroop? En ms, y con un test            | 20 min |
| 5     | Guardar la tabla de efectos                         | 5 min  |

Control de la parte 1: filas = participantes × 36. **Si no da, no sigas.**

# Todos los archivos en una tabla

```
import pandas as pd
from pathlib import Path

archivos = sorted(Path('datos_clase').glob('*.*csv'))
trials = pd.concat([pd.read_csv(a) for a in archivos])

trials.shape      # filas = personas × 36
trials.columns    # ¿de qué grupo es cada una?
trials.head()
```

Una fila por trial, todas las personas apiladas: `participant` dice de quién es cada una.

# Tres filtros de limpieza

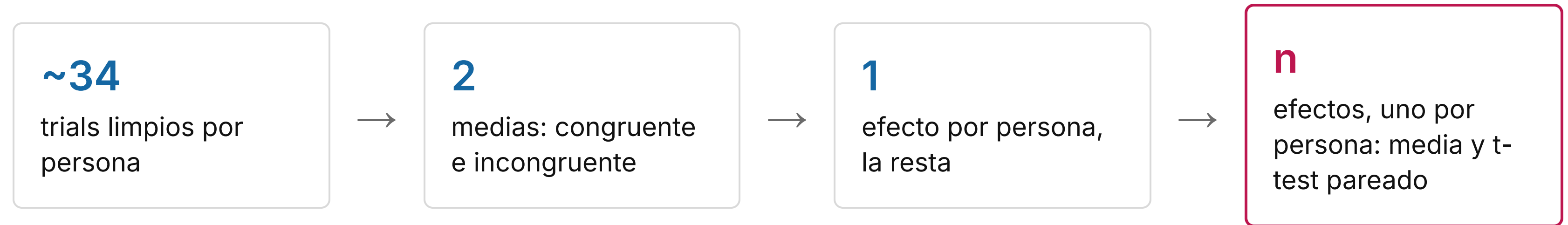
```
es_trial    = trials['response_time'].notna()  
es_correcto = trials['correct'] == 1  
en_ventana  = trials['response_time'].between(0.2, 2.0)  
  
limpios = trials[es_trial & es_correcto & en_ventana]
```

Menos de **0.2 s**: una anticipación,  
no una lectura

Más de **2.0 s**: se distrajo

¿Y por qué esos números?

# Primero dentro de cada persona



$$d_j = \bar{x}_j(\text{incongruente}) - \bar{x}_j(\text{congruente})$$

$$t = \bar{d} / (s / \sqrt{n}), n - 1 \text{ grados de libertad}$$

j: cada participante · s: desvío estándar de las  $d_j$

```
limpios.groupby(['participant', 'congruent'])['response_time'].mean()
```

JUAN

# El efecto Stroop de la clase

— ms

efecto medio, incongruente  
menos congruente

— / —

participantes con efecto  
positivo

$t(\text{—}) = \text{—}$

$p = \text{—}$

En la literatura, entre 70 y 100 ms.

# ¿Cuánto depende de los umbrales?

| Ventana de RT                   | Trials descartados | Efecto |
|---------------------------------|--------------------|--------|
| Sin ventana, solo errores       | —                  | — ms   |
| <b>0.2 a 2.0 s</b> , la nuestra | —                  | — ms   |
| 0.15 a 2.5 s                    | —                  | — ms   |
| 0.3 a 1.5 s                     | —                  | — ms   |

El criterio se decide **antes** de ver el resultado, y se escribe.

# Qué va en Métodos

| Qué       | Ejemplo                                                       |
|-----------|---------------------------------------------------------------|
| Software  | PsychoPy 2026.2.3 (Peirce et al., 2019)                       |
| Pantalla  | Tamaño, resolución, 60 Hz, a 60 cm                            |
| Estímulos | Tamaño en grados, colores, duraciones en ms                   |
| Diseño    | 9 combinaciones de palabra y color, 4 veces cada una, al azar |
| Exclusión | Errores y RT fuera de 0.2–2.0 s, decidido antes               |
| Timing    | Si se validó, y con qué instrumento                           |

# Charla sobre sus proyectos

## Las preguntas de ayer

1. La pregunta, en una frase
2. Un trial, con sus tiempos
3. Lo que varía y lo que se mide
4. Diseño y dónde corre
5. Hardware
6. **El obstáculo**

## Cómo lo hacemos

Con quien quiera, de a un proyecto

Primero EEG, fMRI y eyetracking, y quienes vienen de lejos

Las dudas que se repiten, al pizarrón

CIERRE

# Para seguir

|                   |                                                                                               |
|-------------------|-----------------------------------------------------------------------------------------------|
| <b>Materiales</b> | <a href="https://github.com/joctavio287/DyECCSAN2026">github.com/joctavio287/DyECCSAN2026</a> |
|-------------------|-----------------------------------------------------------------------------------------------|

---

|                  |                                                                     |
|------------------|---------------------------------------------------------------------|
| <b>Preguntas</b> | <a href="https://discourse.psychopy.org">discourse.psychopy.org</a> |
|------------------|---------------------------------------------------------------------|

---

|                  |                                                   |
|------------------|---------------------------------------------------|
| <b>Para leer</b> | recursos/recursero.md: papers, paradigmas, videos |
|------------------|---------------------------------------------------|

---

|                   |                                                                                                                     |
|-------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>Cómo citar</b> | Peirce et al. (2019). PsychoPy2: Experiments in behavior made easy. <i>Behavior Research Methods</i> , 51, 195–203. |
|-------------------|---------------------------------------------------------------------------------------------------------------------|

---

**¡Gracias!**